

Theater Ticketing System

Software Requirements Specification

Version 2

9/24/2026

Group #19

Zachary Gass

Ryoka Kobayashi

Prepared for

CS 250- Introduction to Software Systems

Instructor: Gus Hanna, Ph.D.

Fall 2023

Revision History

Date	Description	Author	Comments
<date>	<Version 1>	<Your Name>	<First Revision>
		Zachary Gass	
		Ryoka Kobayashi	

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

REVISION HISTORY	ii
DOCUMENT APPROVAL	ii
1. INTRODUCTION	1
1.1 PURPOSE	1
1.2 SCOPE	1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	1
1.4 REFERENCES	1
1.5 OVERVIEW	1
2. GENERAL DESCRIPTION	2
2.1 PRODUCT PERSPECTIVE	2
2.2 PRODUCT FUNCTIONS	2
2.3 USER CHARACTERISTICS	2
2.4 GENERAL CONSTRAINTS	2
2.5 ASSUMPTIONS AND DEPENDENCIES	2
3. SPECIFIC REQUIREMENTS	2
3.1 EXTERNAL INTERFACE REQUIREMENTS	3
3.1.1 <i>User Interfaces</i>	3
3.1.2 <i>Hardware Interfaces</i>	3
3.1.3 <i>Software Interfaces</i>	3
3.1.4 <i>Communications Interfaces</i>	3
3.2 FUNCTIONAL REQUIREMENTS	3
3.2.1 <i><Functional Requirement or Feature #1></i>	3
3.2.2 <i><Functional Requirement or Feature #2></i>	3
3.3 USE CASES	3
3.3.1 <i>Use Case #1</i>	3
3.3.2 <i>Use Case #2</i>	3
3.4 CLASSES / OBJECTS	3
3.4.1 <i><Class / Object #1></i>	3
3.4.2 <i><Class / Object #2></i>	3
3.5 NON-FUNCTIONAL REQUIREMENTS	4
3.5.1 <i>Performance</i>	4
3.5.2 <i>Reliability</i>	4
3.5.3 <i>Availability</i>	4
3.5.4 <i>Security</i>	4
3.5.5 <i>Maintainability</i>	4
3.5.6 <i>Portability</i>	4
3.6 INVERSE REQUIREMENTS	4
3.7 DESIGN CONSTRAINTS	4
3.8 LOGICAL DATABASE REQUIREMENTS	4
3.9 OTHER REQUIREMENTS	4
4. ANALYSIS MODELS	4
4.1 SEQUENCE DIAGRAMS	5
4.3 DATA FLOW DIAGRAMS (DFD)	5
4.2 STATE-TRANSITION DIAGRAMS (STD)	5
5. CHANGE MANAGEMENT PROCESS	5
A. APPENDICES	5
A.1 APPENDIX 1	5
A.2 APPENDIX 2	5

1. Introduction

1.1 Purpose

This SRS describes what the Theater Ticketing System needs to do and the limits it has to work within. The requirements come from interviews with our client, the theater chain's management, and each one is written so it can be checked during testing.

The document is written for:

- The client(theater management), to review the requirements and point out anything missing or incorrect.
- Designers and developers, to use as the basis for building the system.
- Testers, to write test cases against each requirement.
- Project managers and the course instructor, to keep track of scope and progress.

1.2 Scope

The product covered here is the Theater Ticketing System. It will replace the ticketing software the chain uses today. The current system does sell tickets, but it is slow, and bugs from past updates have built up to the point where they are hard to manage.

Customers will be able to buy tickets online through a web browser or in person at kiosks in each theater. Both use the same live data, so a seat sold at a kiosk is immediately unavailable on the website, and prices match everywhere. Main features include:

- Browsing movies and showtimes, picking seats, and buying up to 20 tickets in one transaction
- Discounted tickets for students and military members
- Several payment options, with tickets sent digitally or printed
- Optional customer accounts that keep purchase history and loyalty points
- Unique tickets that can't be duplicated, to cut down on scalping
- An administrator mode for setting showtimes, moving movies between theaters, and fixing purchase mistakes
- A short feedback prompt after checkout, plus reviews and critic scores shown alongside each movie

A mobile app is not part of this project; the system is browser-based only. A feature that suggests nearby theaters is also out of scope for now, though the client may revisit it in a later release.

Tickets are only valid at the chain's San Diego locations.

The client's main priorities are a quicker and more dependable way for customers to buy tickets, fair access to seats for popular releases, and better reporting on ticket sales and pricing for each movie. They want a prototype ready for user testing within four months, with a budget of around \$500,000.

1.3 Definitions, Acronyms, and Abbreviations

Administrator - Theater staff with higher access, who manage showtimes, theaters, and correct purchase errors.

Bot - Automated software used to buy large numbers of tickets to resell them.

Deluxe theater - A theater with premium seating and a smaller capacity than a regular theater.

Kiosk - A self-service ticket terminal in a theater lobby.

Loyalty account - An optional customer account that stores contact details, saved payment methods, past purchases, and loyalty points for rewards.

Moviegoer - A customer buying tickets through the system.

Scalping - Buying tickets in bulk to resell them at a markup.

Seat hold - A short reservation on selected seats while the customer finishes checking out.

Showtime - A scheduled screening of a movie in a particular theater. Also called a showing.

IEEE - Institute of Electrical and Electronics Engineers.

SRS - Software Requirements Specification.

UI - User Interface.

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers, 1998.
- W. Y. Arms, “Scenarios and Use Cases,” CS 5150 Software Engineering lecture notes, Cornell University.
- Software Requirements Specification Template, CS 250 Introduction to Software Systems, Dr. Gus Hanna.
- Theater Ticketing System client interview notes, CS 250 in-class client interviews. Available from the authors.

1.5 Overview

The rest of this SRS is organized as follows:

- Section 2, General Description: Background on the product, its users, and the constraints and assumptions behind the requirements. This section is written for a non-technical reader and easy to understand.
- Section 3, Specific Requirements: The detailed requirements, including interfaces, functional requirements, use cases, and non-functional requirements such as performance and security. Requirements are numbered so they can be traced into design and testing.
- Section 4, Analysis Models: Sequence, data flow, and state transition diagrams.
- Section 5, Change Management Process: How changes to this document are requested and approved.
- Appendices: Supporting material such as client interview notes.

2. General Description

2.1 Product Perspective

The Theater Ticketing System will replace the existing ticketing software used by the chain's 20 San Diego theaters. The current system sells tickets, but it is slow, and bugs from past updates have become hard to manage.

With the new system, customers can buy tickets in two ways: only through a web browser, or in person at kiosks in each theater. Both use the same live data, so a seat sold at a kiosk is immediately unavailable on the website, and prices are the same everywhere. Tickets are only valid at the chain's San Diego locations.

The new system is intended to provide a faster and more reliable ticket-buying experience while keeping ticket information consistent between online and in-person sales.

2.2 Product Functions

The main functions of the system are:

- Browse movies and showtimes
- Pick seats and buy up to 20 tickets in one transaction
- Get discounted tickets for students and military member
- Pay with several payment options
- Receive tickets digitally or as printed tickets
- Create an optional customer account that keeps purchase history and loyalty points
- Get unique tickets that can't be duplicated, to cut down on scalping
- Give customers fair access to seats for popular releases
- Let administrators set showtimes, move movies between theaters, and fix purchase mistakes
- Answer a short feedback prompt after checkout, and see reviews and critic scores for each movie
- Provide reporting on ticket sales and pricing for each movie.

2.3 User Characteristics

The system has two types of users:

- Customers: people who buy tickets online or at kiosks. Some are students or military members who qualify for discounted tickets. They can choose whether or not to create a loyalty account.
- Administrators: Theater staff with higher access, who manage showtimes and theater and correct purchase errors.

2.4 General Constraints

The following constraints limit the system's design options:

- The system will only run on the web browser; a mobile app will not be developed.

- The system is entirely local, only being applicable to the San Diego theatre locations.
- One transaction has a hard limit of 20 tickets.
- Selected seats are temporarily reserved in checkout, unavailable to other online and in person buyers until the transaction is completed, or 5 minutes have passed.
- Each ticket must be unique.
- The system must support multiple languages, including English, Spanish, and Swedish.
- Website downtime should only happen when necessary, and kept to a minimum.
- Must be able to handle up to 10 million users simultaneously.
- Tickets can go on sale up to 2 weeks before the showing, and 10 mins after it has started playing.
- A working prototype must be available in 4 months with a budget of \$500,000.
- Only 1 device can be logged into a customer account at once.
- All theatres share one database.

2.5 Assumptions and Dependencies

The requirements assume and depend on the following. If any change, the document may need to be reworked:

- Customers using the website have access to the internet with a working web browser.
- Each theatre has working kiosks, a good internet connection, and the ability to print tickets.
- Rotten Tomatoes and other critic/review websites continue to stay up and allow for the system to integrate its information.
- An email service is available and able to be automated to send digital confirmations and transaction receipts.
- The current showtimes and seating data is available to be transferred into the new platform's database.
- Third party payment options(such as Bitcoin, credit cards, etc.) remain available and allow their services to be integrated.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- 3.1.1.1 The system shall provide a web browser interface for moviegoers to browse movies and showtimes, pick seats, and buy tickets.
- 3.1.1.2 The system shall provide a kiosk interface in each theater for moviegoers to buy tickets in person.
- 3.1.1.3 The system shall show which seats are available for a selected showtime.
- 3.1.1.4 The system shall be available in multiple languages, including English and Spanish.
- 3.1.1.5 The system shall let moviegoers create an optional customer account and view their purchase history and loyalty points.
- 3.1.1.6 The system shall show a short feedback prompt after checkout.
- 3.1.1.7 The system shall show reviews and critic scores alongside each movie.
- 3.1.1.8 The system shall provide an administrator mode for setting showtimes, moving movies between theaters, and fixing purchase mistakes.

3.1.2 Hardware Interfaces

- 3.1.2.1 The system shall run on kiosks located in each of the chain's theaters.
- 3.1.2.2 The system shall support printed tickets.
- 3.1.2.3 Moviegoers shall be able to use the website from a device with a web browser.

3.1.3 Software Interfaces

- 3.1.3.1 The system shall run in a web browser. A mobile app will not be developed.
- 3.1.3.2 The website and kiosks shall use the same live data, so seat availability and prices match everywhere.
- 3.1.3.3 The system shall accept several payment options.

3.1.4 Communications Interfaces

- 3.1.4.1 When a seat is sold at a kiosk, it shall immediately become unavailable on the website, and when a seat is sold on the website, it shall immediately become unavailable at the kiosks.
- 3.1.4.2 Seats held during checkout shall be unavailable to other online and in-person buyers until the transaction is completed or the hold times out.
- 3.1.4.3 The system shall send tickets to moviegoers digitally.

3.2 Functional Requirements

3.2.1 <Browse Movies and Showtimes>

3.2.1.1 Introduction

Moviegoers shall be able to browse movies and showtimes at the chain's San Diego theaters, online or at a kiosk.

3.2.1.2 Inputs

The theater, movie, and showtime chosen by the moviegoer.

3.2.1.3 Processing

The system shall retrieve movies and showtimes for the chosen from the same live data used by both the website and kiosks.

3.2.1.4 Outputs

A list of movies and their showtimes, with reviews and critic scores shown alongside each movie.

3.2.1.5 Error Handling

If a movie has no available showtimes, the system shall tell the moviegoer.

3.2.2 <Seat Selection and Seat Hold>

3.2.2.1 Introduction

Moviegoers shall be able to pick seats from a seat map. Selected seats are held while the moviegoer checks out.

3.2.2.2 Inputs

The Selected showtime and the seats the moviegoer picks.

3.2.2.3 Processing

The system shall show a seat map for the selected showtime. When the moviegoer picks seats, the system shall hold them for up to 5 minutes. Held seats shall be unavailable to other online and in-person buyers until the transaction is completed or the hold times out.

3.2.2.4 Outputs

The seat map showing available seats, the seats held for the moviegoer, and the total price.

3.2.2.5 Error Handling

Seats that are already sold or held shall not be selectable. If the transaction is not completed within 5 minutes, the hold shall expire and the seats shall become available to other buyers.

3.2.3 <Ticket Purchase>

3.2.3.1 Introduction

Moviegoers shall be able to buy tickets online or at a kiosk. Payments are processed by an outside payment processor.

3.2.3.2 Inputs

The number of tickets, the ticket type for each ticket(adult, student, senior, or military), and the payment method and payment details.

3.2.3.3 Processing

The system shall allow up to 20 tickets in one transaction. It shall apply the price for each ticket type, and prices shall be the same on the website and at kiosks. The system shall send the payment information to the payment processor for authorization. When the payment is authorized, the seats shall become unavailable on both the website and kiosk within 1 second.

3.2.3.4 Outputs

The total price before payment, and a purchase confirmation after payment is authorized.

3.2.3.5 Error Handling

The system shall not allow more than 20 tickets in one transaction. If the payment processor does not authorize the payment, no tickets shall be issued.

3.2.4 <Ticket Delivery>

3.2.4.1 Introduction

The system shall create unique tickets and deliver them to the moviegoer. Unique tickets help cut down on scalping.

3.2.4.2 Inputs

The authorized purchase and whether it was made online or at a kiosk.

3.2.4.3 Processing

The system shall create a unique ticket for each seat purchased. Tickets shall not be able to be duplicated. Tickets for online purchases shall be emailed to the moviegoer. Tickets for kiosk purchases shall be printed within 1 minute of the purchase being confirmed.

3.2.4.4 Outputs

Emailed tickets for online purchases, or printed tickets for kiosk purchases.

3.2.4.5 Error Handling

If a kiosk printer fails or runs out of paper, the kiosk shall use a backup printer or give the moviegoer a digital receipt.

3.2.5 <Customer Accounts>

3.2.5.1 Introduction

Moviegoers may create an optional loyalty account. Moviegoers can buy tickets without one.

3.2.5.2 Inputs

The moviegoer's name, email, and password, and optionally their payment information.

3.2.5.3 Processing

The system shall check that the email is not already used by another account. It shall then create the account and send a confirmation email. After the moviegoer confirms the email, they shall be logged into their account as a member. The account shall keep the moviegoer's contact details, past purchases, and loyalty points. Saved payment information shall not be stored on the system's server.

3.2.5.4 Outputs

A confirmation email, and the moviegoer's purchase history and loyalty points.

3.2.5.5 Error Handling

If the email is already in use, the system shall not create the account and shall tell the moviegoer.

3.2.6 <Administrator Mode>

3.2.6.1 Introduction

Administrators shall be able to manage showtimes, move movies between theaters, and correct purchase errors.

3.2.6.2 Inputs

Administrator login credentials, the selected theater, and the showtime to add, edit, or remove.

3.2.6.3 Processing

The system shall confirm the administrator's credentials before allowing access. It shall show the selected theater's current movies and showtimes. When the administrator adds, edits, or removes a showtime, the system shall check for conflicts, such as overlapping showtimes or showtimes that have already sold tickets. The system shall save the changes and update the website and kiosks within 1 second.

3.2.6.4 Outputs

The theater's current movies and showtimes, and the updated showtimes on the website and kiosks.

3.2.6.5 Error Handling

If the credentials are not valid, the system shall deny access to administrator mode. If a change creates a conflict, the system shall not save the change and shall tell the administrator.

3.2.7 <Feedback, Reviews, and Critic Scores>

3.2.7.1 Introduction

The system shall offer an optional feedback prompt after checkout and show reviews and critic scores for each movie.

3.2.7.2 Inputs

The moviegoer's answers to the feedback prompt, if they choose to answer.

3.2.7.3 Processing

The system shall show the feedback prompt after checkout. The moviegoer may skip it. The system shall store any answers given.

3.2.7.4 Outputs

Reviews and critic scores shown alongside each movie.

3.2.7.5 Error Handling

If the moviegoer skips the feedback prompt, the purchase shall still be completed.

3.2.8 <Sales Reporting>

3.2.8.1 Introduction

The system shall provide reports on ticket sales and pricing for each movie.

3.2.8.2 Inputs

The movie selected for the report.

3.2.8.3 Processing

The system shall gather ticket sales and pricing data for the selected movie.

3.2.8.4 Outputs

A report of ticket sales and pricing for the movie.

3.2.8.5 Error Handling

If there are no sales for the selected movie, the report shall show zero sales.

3.3 Use Cases

3.3.1 Use Case #1- Purchase Tickets

Actor: Customer

Event flow:

- Customer goes to the in person kiosk or opens the website
- Customer chooses a theatre, movie, and showtime
- System shows a seatmap for the specified showing
- Customer chooses up to 20 tickets and chooses the types(adult, student, senior, military)

- System reserves the seats for up to 5 minutes and displays the total
- Customer chooses a payment method and enters the details
- System sends the information to the payment processor
- Payment processor authorizes the payment
- System creates unique tickets and either emails them or prints them for pickup, for online and in person purchases respectively
- System displays an optional feedback prompt

3.3.2 Use Case #2- Register Account

Actor: Customer

Event Flow:

- Customer selects the option to create the account
- System prompts the user for their name, email, and a password to create the account
- Customer enters the information, with an option to save payment information
- System confirms the email isn't already in use
- System creates an account and sends a confirmation email
- Customer confirms the email and is logged into their account as a member

Use Case #3- Manage Showtimes

Actor: Administrator

Event Flow:

- Administrator logs into an Administrator account
- System confirms the Administrator's credentials
- Administrator selects a theatre
- System shows the theatre's current movies and showtimes
- Administrator adds a new showtime, or edits/removes an existing one
- System checks for conflicts, such as overlapping/conflicting show times, or already sold tickets
- System saves the changes and updates the websites and kiosks immediately
- Administrator logs out of the Administrator account

...

3.4 Classes / Objects



3.4.1 <Movie>

3.4.1.1 Attributes

- Title
- Reviews
- Critic scores

3.4.1.2 Functions

- Show the movie's showtimes
- Show reviews and critic score alongside the movie

<Related requirements: 3.2.1, 3.2.7; use case 3.3.1>

3.4.2 <Theater>

3.4.2.1 Attributes

- Location (San Diego)
- Theater type (regular or deluxe)
- Seats
- Current movies and showtimes

3.4.2.2 Functions

- Show the theater's current movies and showtimes

<Related requirements:3.2.1, 3.2.6; use case 3.3.1, 3.3.3>

3.4.3 <Showtime>

3.4.3.1 Attributes

- Movie
- Theater
- Time
- Seat Availability

3.4.3.2 Functions

- Show a seat map
- Check for conflicts with other showtimes and sold tickets when added, edited, or removed

<Related requirements:3.2.1, 3.2.2, 3.2.6; use case 3.3.1, 3.3.3>

3.4.4 <Seat>

3.4.4.1 Attributes

- Seat number
- Status (available, held, or sold)
- Hold expiration time

3.4.4.2 Functions

- Hold the seat for up to 5 minutes
- Release the seat when the hold times out
- Mark the seat as sold

<Related requirements:3.2.2, 3.2.3; use case 3.3.1>

3.4.5 <Purchase>

3.4.5.1 Attributes

- Tickets (up to 20)
- Ticket types (adult, student, senior, or military)
- Total price
- Payment method
- Purchase location(online or kiosk)

3.4.5.2 Functions

- Calculate the total price
- Send payment information to the payment processor
- Confirm the purchase after payment is authorized

<Related requirements: 3.2.3; use case 3.3.1>

3.4.6 <Ticket>

3.4.6.1 Attributes

- Unique tickets ID
- Showtime
- Seat
- Ticket type
- Price

3.4.6.2 Functions

- Create a unique ticket that cannot be duplicated
- Email the ticket (online purchases)
- Print the ticket (kiosk purchases)

<Related requirements:3.2.4; use case 3.3.1>

3.4.7 <CustomerAccount>

3.4.7.1 Attributes

- Name
- Password
- Purchase history
- Loyalty points

3.4.7.2 Functions

- Check that the email is not already in use
- Create the account and send a confirmation email
- Show purchase history and loyalty points

<Related requirements:3.2.5; use case 3.3.2>

3.4.8 <Seat>

3.4.8.1 Attributes

- Login credentials

3.4.8.2 Functions

- Log in and log out of administrator mode
- Add, edit, or remove showtimes
- Move movies between theaters
- Fix purchase mistakes

<Related requirements:3.2.6; use case 3.3.3>

3.4.9 <Feedback>

3.4.9.1 Attributes

- Moviegoer's answers to the feedback prompt

3.4.9.2 Functions

- Store the answers given after checkout

<Related requirements:3.2.7; use case 3.3.1>

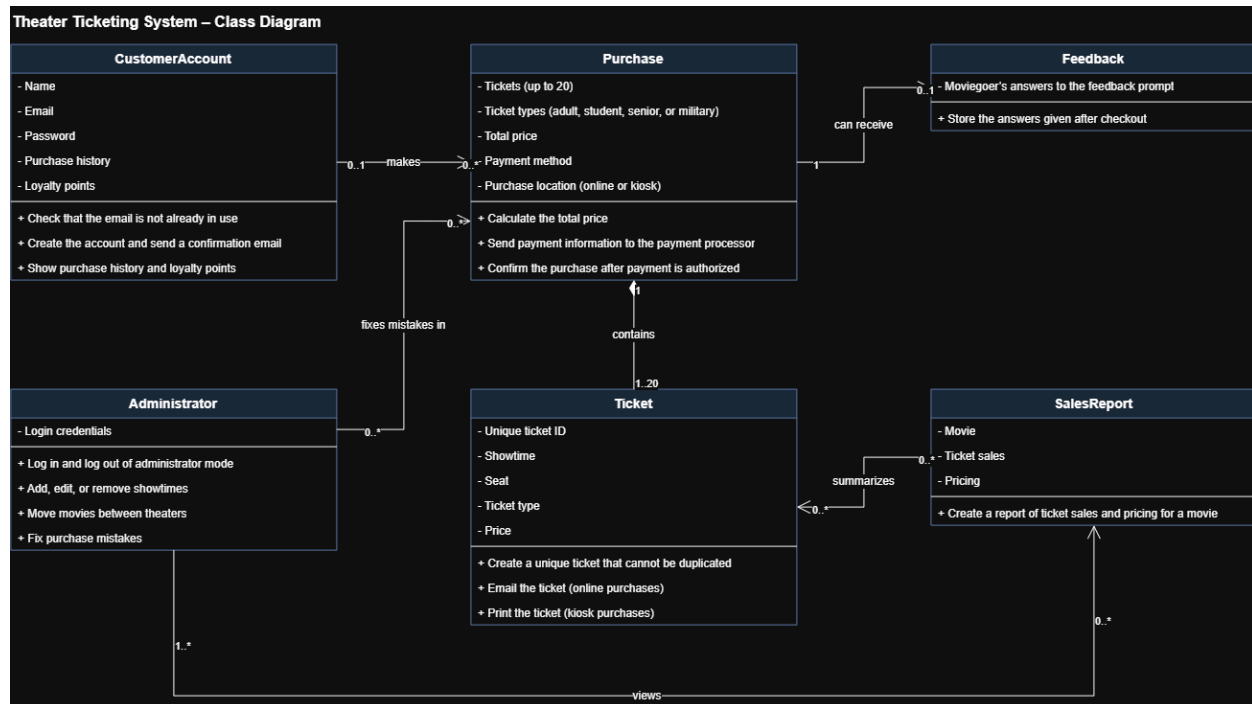
3.4.10 <Sales Report>

3.4.10.1 Attributes

- Movie
- Ticket Sales
- Pricing

3.4.10.2 Functions

- Create a report of ticket sales and pricing for a movie



<Related requirements:3.2.8>

3.5 Non-Functional Requirements

3.5.1 Performance

- 95% of transactions must be processed in less than 3 seconds, only varying on the customer's own connection speed
- The system should load new pages(such as seat viewing, theatre selection, ticket selection, as well as account creation and logging in) in less than 1 second
- Updates from the system to the kiosk and the website should be processed within less than a second- seats sold should appear as reserved within 2 seconds of the reservation
- Tickets should be printed within the minute the purchase is confirmed by the system; customers waiting for over 5 minutes
- The system should be able to support at minimum over 1000 concurrent users without noticeable delay or lag in interactions

3.5.2 Reliability

- Kiosks should have backup printers or offer digital receipts in the events printers fail or run out of printing paper
- System downtime should be kept at a minimum of 1 hour per day, exclusively during periods of low activity, such as overnight hours(12AM -6AM PST)
- If a transaction fails or is interrupted, the customer will not be charged and tickets will not be issued.

- The system will never sell the same seat twice. Once a unique ticket is issued, it can no longer be issued again, and if transactions happen within seconds, the late user will have their purchase refunded if the purchase went through before the system updates.

3.5.3 Availability

- The website should be readily accessible and able to be navigated on any platform that is able to access it; different layouts(such as mobile layouts, desktop, tablets) should all be able to perform the same functions for every user accessing them
- Kiosks must be accessible physically(via ramps, elevators, and flat ground) as an alternative to stairs to persons with disabilities. Purchased tickets should be automatically sent to the customer's associated email, and in person purchases should have receipts and tickets handed to them after purchase by kiosk workers
- Scheduled maintenance will occur within the hours of 2AM - 6AM PST exclusively, where a notification will be posted on the website to warn customers prior. Maintenance will also only be implemented in emergency situations(vulnerabilities, system bugs or errors) when necessary, and will always be done as close to the range of 2AM -6AM PST if possible

3.5.4 Security

- Backup databases of movie showtimes and customer purchase logs should be stored and maintained in the event of a system wide failure, or a 3rd party breach
- Payment information will either be stored locally, or with the customer's 3rd party. No payment information will be stored on the system's server, in the event admins attempt to defraud the government or a breach occurs
- All website data shall be sent of HTTPS
- Each ticket will have a unique token and signature, such that duplicates are rejected; tickets will follow the same identifying structure as NFTs(with a contact address, metadata associated with the ticket and which account purchased it, and a unique token).
- Only one device can be logged into an account at a time
- To prevent scalping, a user canceling a transaction or timing out will be temporarily capped at the total amount of seats they can preserve for 2 minutes, to allow other users and in person users to purchase the seats before they can be reserved by the same person consecutively.
- Passwords will be stored encrypted to prevent unauthorized access
- Card payments will meet PCI DSS standard
- The system will implement a bot detection captcha for account creation, log in, and ticket purchasing for high demand showings

3.5.5 Maintainability

- Server code and implementation should be thoroughly documented so fixes and additions can be easily implemented
- The system should be built off modules, such that changes or maintenance in one department doesn't affect the availability of the others

- Updates shall be processed such that the website isn't fully disabled and still has functionality

3.5.6 Portability

- This system should be able to add new locations without major system reworks or changes
- All kiosks will run the same web application as the websites
- The website shall run on all current versions of Google Chrome, Windows explorer, Safari, Firefox, and Edge, on desktop and mobile devices

3.6 Inverse Requirements

- 3.6.1 The system shall not allow more than 20 tickets in one transaction.
- 3.6.2 The system shall not allow a seat that is held or sold to be selected by another online or in-person buyer.
- 3.6.3 The system shall not issue two tickets for the same seat and showtime, and tickets shall not be able to be duplicated.
- 3.6.4 The system shall not issue tickets if the payment processor does not authorize the payment.
- 3.6.5 The system shall not charge different prices for the same ticket on the website and at kiosks.
- 3.6.6 The system shall not require moviegoers to create an account to buy tickets.
- 3.6.7 The system shall not require moviegoers to answer the feedback prompt to complete a purchase.
- 3.6.8 The system shall not store payment information on the system's server.
- 3.6.9 The system shall not allow users without administrator access to use administrator mode.
- 3.6.10 The system shall not create an account with an email that is already in use.
- 3.6.11 The system shall not save showtime changes that conflict with other showtimes or with already sold tickets.
- 3.6.12 The system shall not include a mobile app or a feature that suggests nearby theaters.
- 3.6.13 Tickets shall not be valid at locations outside the chain's San Diego theaters.

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

6. Software Design Specification

This section describes how the Theater Ticketing System will be built. Sections 1 through 5 describe what the system needs to do and are written with the client in mind. This section is written for the developers who will implement the system and maintain it afterward. It covers the overall software architecture, the classes that make up the design, and the plan for building it.

6.1 System Description

The Theater Ticketing System is a web-based system for selling movie tickets at a chain of 20 theaters in San Diego. Customers can buy tickets on the website or at kiosks inside each theater, and both use the same live data, so a seat sold in one place is immediately unavailable everywhere else.

Customers browse movies and showtimes, pick their seats, choose a ticket type for each seat (adult, student, senior, or military), and pay by credit card, PayPal, or Bitcoin. Up to 20 tickets can be bought in one transaction. Every ticket is unique and digitally signed so it cannot be copied. Tickets are e-mailed for online orders and printed for kiosk orders. Customers can create an optional account to keep their purchase history and earn loyalty points, and they are asked for quick feedback after checkout.

Theater staff use an administrator mode to add and change showtimes, move movies between theaters, fix purchase mistakes, and view ticket sales reports.

6.2 Software Architecture Overview

The architecture is shown in Figure 6.1 on the next page, followed by a description of each component and connector. The UML class diagram is in Section 6.3, and each class is described in Section 6.4.

6.2.1 Software Architecture Diagram

Theater Ticketing System – Software Architecture Diagram

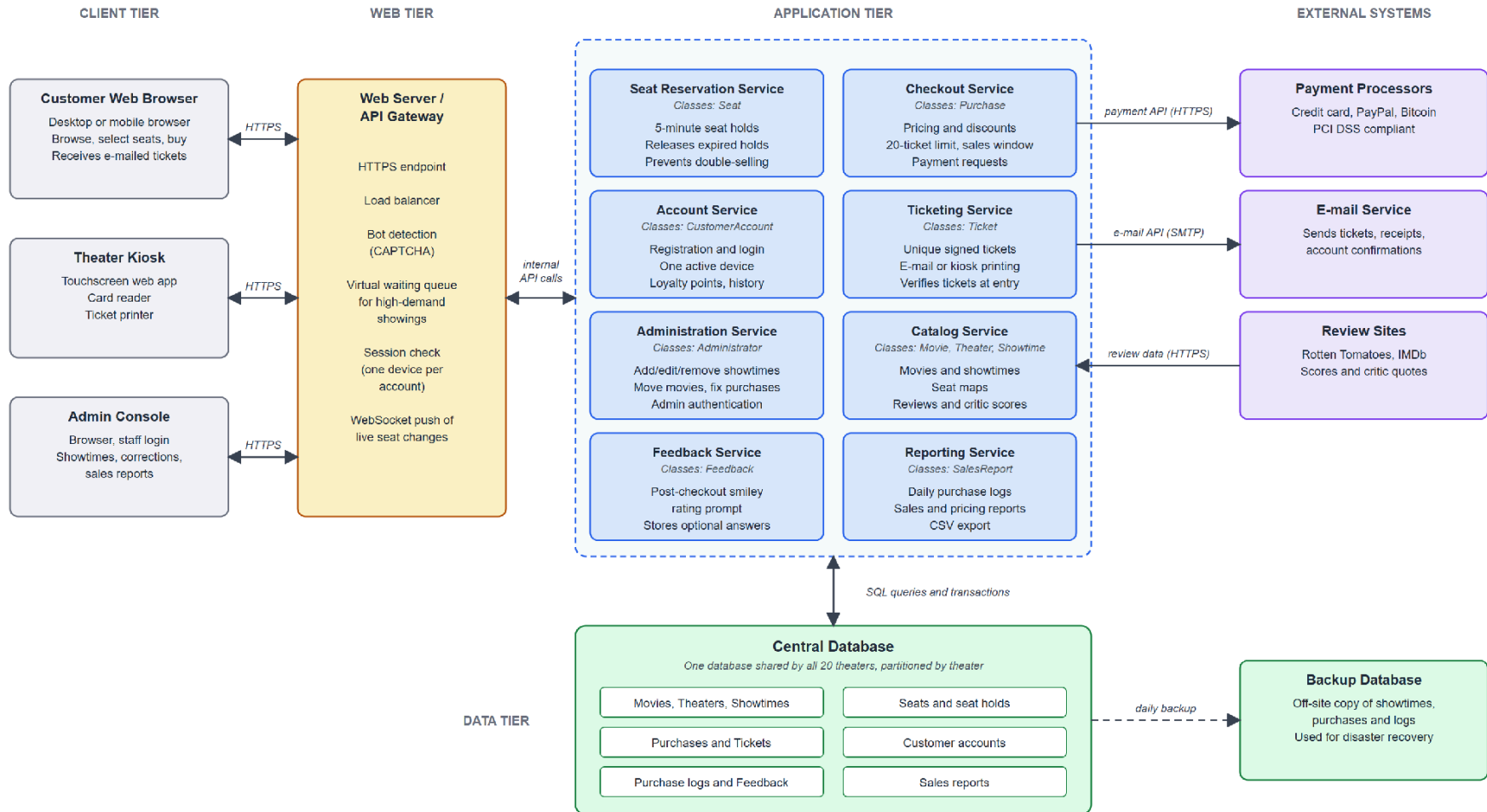


Figure 6.1 – Software architecture of the Theater Ticketing System

6.2.2 Software Architecture Description

The system uses a layered, service-based architecture. Requests from the website, the kiosks, and the admin console all go through one web server, which passes them to a set of application services. Each service handles one area of the system and is built from the classes named in its box in Figure 6.1. All services read from and write to one central database. Because each service is separate, one can be updated or restarted without taking down the rest of the site, which supports the maintainability and availability requirements in Sections 3.5.3 and 3.5.5.

Client tier

- **Customer Web Browser:** The public website. It runs in any current desktop or mobile browser, so customers do not need to install an app.
- **Theater Kiosk:** A touchscreen terminal in each theater lobby. It runs the same web application as the website, with access to the kiosk's card reader and ticket printer.
- **Admin Console:** The administrator side of the website. Staff have to log in with administrator credentials before they can use it.

Web tier

- **Web Server / API Gateway:** The single entry point for all client traffic. It handles HTTPS, spreads load across servers, and runs CAPTCHA checks to block bots. When a showing is in high demand, it places customers in a virtual waiting queue and lets them in first-come, first-served. It also enforces the one-device-per-account rule and pushes seat changes to open browsers and kiosks over a WebSocket connection, so seat maps stay current without the customer refreshing the page.

Application tier

- **Seat Reservation Service (Seat):** Places 5-minute holds on selected seats, releases holds that expire, and makes sure a seat can only be held or sold by one customer at a time.
- **Checkout Service (Purchase):** Builds the order, applies the price for each ticket type, enforces the 20-ticket limit and the sales window, and sends payment requests to the payment processor.
- **Ticketing Service (Ticket):** Creates a unique, signed ticket for each seat once payment is approved, e-mails or prints it, and checks tickets when they are scanned at the theater entrance.
- **Catalog Service (Movie, Theater, Showtime):** Provides movies, showtimes, and seat maps. It also pulls scores and critic quotes from review sites and caches them, so a review site being down does not stop movie pages from loading.
- **Account Service (CustomerAccount):** Handles registration, login and logout, loyalty points, and purchase history.
- **Administration Service (Administrator):** Handles administrator login, showtime changes, moving movies between theaters, and purchase corrections. It checks for conflicts before saving any change.
- **Feedback Service (Feedback):** Shows the rating prompt after checkout and stores any answers.

- **Reporting Service (SalesReport):** Writes every purchase to the daily log and builds ticket sales and pricing reports for administrators.

Data tier

- **Central Database:** One relational database shared by all 20 theaters and partitioned by theater. It stores movies, showtimes, seats, purchases, tickets, accounts, logs, and feedback. Seat holds and purchases are written inside database transactions, so the same seat can never be sold twice.
- **Backup Database:** An off-site copy of the central database, updated daily. It is used to restore data after a system failure.

External systems

- **Payment Processors:** Outside services that handle credit card, PayPal, and Bitcoin payments. Payment details go straight from the customer to the processor; the system only receives a payment token and the result. This keeps payment information off the system's servers (Section 3.6.8) and meets PCI DSS.
- **E-mail Service:** Sends tickets, receipts, and account confirmation e-mails.
- **Review Sites:** Rotten Tomatoes and IMDb, which supply review scores and critic quotes.

Connectors

Connector	Connects	Description
HTTPS (REST and WebSocket)	Browser, kiosk, and admin console ↔ web server	Encrypted requests and responses. The WebSocket connection pushes live seat changes to the client.
Internal API calls	Web server ↔ application services	The web server forwards each request to the service responsible for it.
SQL queries and transactions	Application services ↔ central database	Reads and writes all system data. Seat holds and purchases use transactions so they either fully succeed or have no effect.
Payment API (HTTPS)	Checkout Service → payment processors	Sends the order total and payment token and receives an approved or declined result.
E-mail API (SMTP)	Ticketing and Account services → e-mail service	Sends tickets, receipts, and confirmation e-mails.
Review data (HTTPS)	Review sites → Catalog Service	Review scores and quotes are refreshed regularly and cached in the database.
Daily backup	Central database → backup database	Copies data off-site once a day during low-traffic hours.

6.3 UML Class Diagram

Figure 6.2 on the next page shows the classes in the system, their attributes and operations, and how they are connected. Section 6.4 describes each class.

Theater Ticketing System – UML Class Diagram

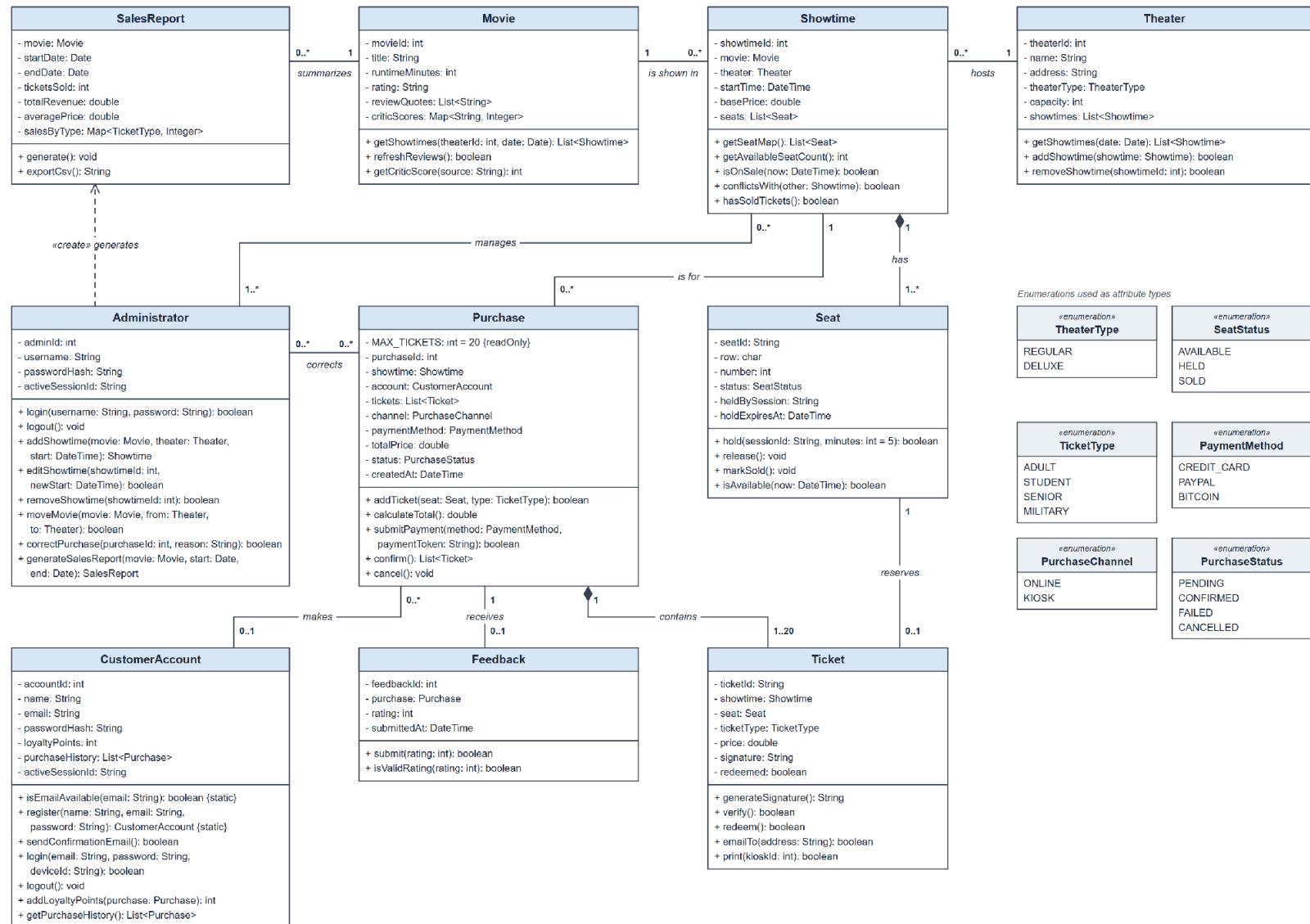


Figure 6.2 – UML class diagram of the Theater Ticketing System

6.4 Description of Classes

Each class in Figure 6.2 is described below with its attributes and operations. Datatypes use Java-style names: int, double, boolean, char, String, Date (a calendar day), DateTime (a date and time), List<T> (an ordered list of T), and Map<K, V> (a lookup from K to V). In the diagram, “-” marks a private attribute and “+” marks a public operation. Attributes are only changed through the class’s operations.

Several attributes use the enumerations below, which limit them to a fixed set of values.

Enumeration	Values	Used by
TheaterType	REGULAR, DELUXE	Theater.theaterType
SeatStatus	AVAILABLE, HELD, SOLD	Seat.status
TicketType	ADULT, STUDENT, SENIOR, MILITARY	Ticket.ticketType, SalesReport.salesByType
PaymentMethod	CREDIT_CARD, PAYPAL, BITCOIN	Purchase.paymentMethod
PurchaseChannel	ONLINE, KIOSK	Purchase.channel
PurchaseStatus	PENDING, CONFIRMED, FAILED, CANCELLED	Purchase.status

6.4.1 Movie

A film the chain is showing. It stores the movie’s basic details along with the review scores and critic quotes pulled from review sites.

Related requirements: 3.2.1, 3.2.7; use case 3.3.1

Attributes

Attribute	Type	Description
movieId	int	Unique ID for the movie.
title	String	Movie title shown to customers.
runtimeMinutes	int	Length of the movie in minutes. Used when checking showtimes for overlaps.
rating	String	Content rating, such as “PG-13”.
reviewQuotes	List<String>	Short critic quotes shown on the movie’s page.
criticScores	Map<String, Integer>	Score from each review site, keyed by site name (for example, “Rotten Tomatoes” → 92).

Operations

Operation	Parameters	Returns	Description
getShowtimes	theaterId: int date: Date	List<Showtime>	Returns this movie’s showtimes at the given theater on the given date, ordered by start time.

Operation	Parameters	Returns	Description
refreshReviews	none	boolean	Pulls the latest scores and quotes from the review sites. Returns false if a site cannot be reached; the previous values are kept so the movie page still loads.
getCriticScore	source: String	int	Returns the score from one review site, or -1 if that site has no score for the movie.

6.4.2 Theater

One of the chain's 20 theaters. Each theater is either a regular theater with 150 seats or a deluxe theater with 75 seats, and keeps its own list of showtimes.

Related requirements: 3.2.1, 3.2.6; use cases 3.3.1, 3.3.3

Attributes

Attribute	Type	Description
theaterId	int	Unique ID for the theater.
name	String	Theater name shown to customers.
address	String	Street address in San Diego.
theaterType	TheaterType	REGULAR or DELUXE.
capacity	int	Number of seats: 150 for regular theaters, 75 for deluxe theaters.
showtimes	List<Showtime>	All showtimes scheduled at this theater.

Operations

Operation	Parameters	Returns	Description
getShowtimes	date: Date	List<Showtime>	Returns the showtimes scheduled on the given date, ordered by start time.
addShowtime	showtime: Showtime	boolean	Adds a showtime if it does not overlap an existing one (checked with Showtime.conflictsWith). Returns false and makes no change if there is a conflict.
removeShowtime	showtimeId: int	boolean	Removes a showtime. Returns false if the showtime is not found or already has tickets sold.

6.4.3 Showtime

A single screening of one movie in one theater at a set time. Each showtime owns the seats for that screening.

Related requirements: 3.2.1, 3.2.2, 3.2.6; use cases 3.3.1, 3.3.3

Attributes

Attribute	Type	Description
showtimeId	int	Unique ID for the showtime.
movie	Movie	The movie being shown.
theater	Theater	The theater it is shown in.
startTime	DateTime	Date and time the movie starts (Pacific time).
basePrice	double	Adult ticket price in US dollars. Discounts for other ticket types are applied to this price.
seats	List<Seat>	One Seat object for each seat in the theater, created along with the showtime.

Operations

Operation	Parameters	Returns	Description
getSeatMap	none	List<Seat>	Returns every seat with its current status, used to draw the seat map.
getAvailableSeatCount	none	int	Returns how many seats are still available.
isOnSale	now: DateTime	boolean	Returns true if tickets can be sold at the given time: from 2 weeks before startTime until 10 minutes after it.
conflictsWith	other: Showtime	boolean	Returns true if the other showtime is in the same theater and the two overlap, based on each movie's runtime.
hasSoldTickets	none	boolean	Returns true if any seat is SOLD. Checked before a showtime is edited or removed.

6.4.4 Seat

One seat for a specific showtime. It tracks whether the seat is available, held during checkout, or sold.

Related requirements: 3.2.2, 3.2.3; use case 3.3.1

Attributes

Attribute	Type	Description
seatId	String	Row letter and seat number, such as "F12".
row	char	Row letter.
number	int	Seat number within the row.
status	SeatStatus	AVAILABLE, HELD, or SOLD.
heldBySession	String	Session ID of the customer holding the seat. Empty when the seat is not held.

Attribute	Type	Description
holdExpiresAt	DateTime	Time the current hold ends. Empty when the seat is not held.

Operations

Operation	Parameters	Returns	Description
hold	sessionId: String minutes: int = 5	boolean	Marks the seat HELD for the given session. The hold lasts 5 minutes unless another value is passed. Returns false if the seat is not available. Runs as one database transaction so two customers cannot hold the same seat.
release	none	void	Sets the seat back to AVAILABLE and clears the hold. Called when a hold expires or a purchase is cancelled.
markSold	none	void	Sets the seat to SOLD after payment is approved.
isAvailable	now: DateTime	boolean	Returns true if the seat is AVAILABLE, or if it is HELD but the hold has already expired.

6.4.5 Purchase

One ticket buying transaction, made either online or at a kiosk. It groups the tickets being bought, works out the total, and handles payment. Customers do not need an account, so the account attribute is empty for guest purchases.

Related requirements: 3.2.3; use case 3.3.1

Attributes

Attribute	Type	Description
MAX_TICKETS	int (constant = 20)	Most tickets allowed in one purchase. Cannot be changed.
purchaseId	int	Unique ID for the purchase.
showtime	Showtime	The showtime the tickets are for.
account	CustomerAccount	The customer's account, or empty for a guest purchase.
tickets	List<Ticket>	The tickets in this purchase (1 to 20).
channel	PurchaseChannel	ONLINE or KIOSK. Decides whether tickets are e-mailed or printed.
paymentMethod	PaymentMethod	CREDIT_CARD, PAYPAL, or BITCOIN.
totalPrice	double	Total cost in US dollars after discounts.
status	PurchaseStatus	PENDING, CONFIRMED, FAILED, or CANCELLED.

Attribute	Type	Description
createdAt	DateTime	Time the purchase was started.

Operations

Operation	Parameters	Returns	Description
addTicket	seat: Seat type: TicketType	boolean	Adds a ticket for a seat this customer is holding. Returns false if the purchase already has 20 tickets or the seat is not held by this customer.
calculateTotal	none	double	Adds up the price of each ticket after its discount, stores the result in totalPrice, and returns it.
submitPayment	method: PaymentMethod paymentToken: String	boolean	Sends the total and payment token to the payment processor. Returns true if the payment is approved. If it is declined, the purchase stays PENDING and the customer can try another method while the seat hold lasts.
confirm	none	List<Ticket>	Called after payment is approved. Sets the status to CONFIRMED, marks each seat SOLD, creates the signed tickets, and writes the purchase to the daily log. Returns the tickets.
cancel	none	void	Releases all held seats and sets the status to CANCELLED. The customer is not charged.

6.4.6 Ticket

Admission for one seat at one showtime. Each ticket has a unique ID and a digital signature, so copied or altered tickets are rejected at the entrance.

Related requirements: 3.2.4; use case 3.3.1

Attributes

Attribute	Type	Description
ticketId	String	Unique ID (a randomly generated UUID).
showtime	Showtime	The showtime the ticket is for.
seat	Seat	The seat the ticket is for.
ticketType	TicketType	ADULT, STUDENT, SENIOR, or MILITARY.
price	double	Price paid for this ticket after its discount.
signature	String	Digital signature created from the ticket ID, showtime, and seat.

Attribute	Type	Description
redeemed	boolean	True once the ticket has been scanned at the entrance.

Operations

Operation	Parameters	Returns	Description
generateSignature	none	String	Creates the ticket's signature using the system's private key. Called once, when the purchase is confirmed.
verify	none	boolean	Returns true if the signature is valid and the ticket has not been redeemed.
redeem	none	boolean	Marks the ticket as used when it is scanned at the entrance. Returns false if it was already used, so a copy of a ticket only works once.
emailTo	address: String	boolean	E-mails the ticket, with a scannable code, through the e-mail service. Returns false if sending fails.
print	kioskId: int	boolean	Prints the ticket at the given kiosk. Returns false on a printer error, in which case the kiosk uses its backup printer or gives a digital receipt (Section 3.2.4.5).

6.4.7 CustomerAccount

An optional loyalty account for a moviegoer. It stores the customer's contact details, purchase history, and loyalty points. Customers can buy tickets without one, so a Purchase may have no account. Passwords are never stored in plain text, and no payment information is stored in this class (Section 3.6.8).

Related requirements: 3.2.5, 3.5.4; use case 3.3.2

Attributes

Attribute	Type	Description
accountId	int	Unique ID for the account.
name	String	Customer's full name.
email	String	Customer's e-mail address. Used to log in and to receive tickets. Must be unique across all accounts.
passwordHash	String	Hashed (encrypted) form of the password. The plain password is

Attribute	Type	Description
		never stored.
loyaltyPoints	int	Current loyalty point balance. Starts at 0.
purchaseHistory	List<Purchase>	All confirmed purchases made with this account, newest first.
activeSessionId	String	Session ID of the one device currently logged in. Empty when no device is logged in.

Operations

Operation	Parameters	Returns	Description
isEmailAvailable {static}	email: String	boolean	Returns true if no existing account uses the given e-mail. Called before an account is created.
register {static}	name: String email: String password: String	CustomerAccount	Creates a new account if the e-mail is available, stores a hash of the password, and sends a confirmation e-mail. Returns the new account, or null if the e-mail is already in use (Section 3.2.5.5).
sendConfirmationEmail	none	boolean	Sends a confirmation link to the account's e-mail through the e-mail service. Returns false if sending fails.
login	email: String password: String deviceId: String	boolean	Checks the password against passwordHash. If correct, starts a new session for the device and stores it in activeSessionId. Any other device logged into the account is logged out, so only one device is logged in at a time.

Operation	Parameters	Returns	Description
			Returns false if the e-mail or password is wrong.
logout	none	void	Ends the current session and clears activeSessionId.
addLoyaltyPoints	purchase: Purchase	int	Adds points for a confirmed purchase (1 point per whole dollar of totalPrice) and adds the purchase to purchaseHistory. Returns the new point balance.
getPurchaseHistory	none	List<Purchase>	Returns the account's past purchases, newest first.

6.4.8 Administrator

A theater staff member with higher access. Administrators manage showtimes, move movies between theaters, correct purchase mistakes, and create sales reports. Every operation except login requires the administrator to be logged in.

Related requirements: 3.2.6, 3.2.8, 3.6.9, 3.6.11; use case 3.3.3

Attributes

Attribute	Type	Description
adminId	int	Unique ID for the administrator.
username	String	Name used to log in to administrator mode.
passwordHash	String	Hashed (encrypted) form of the password.
activeSessionId	String	Session ID of the current administrator login. Empty when logged out.

Operations

Operation	Parameters	Returns	Description
login	username: String password: String	boolean	Checks the credentials and starts an administrator session. Returns false and denies access if the credentials are not valid (Section 3.2.6.5).
logout	none	void	Ends the administrator session and clears activeSessionId.
addShowtime	movie: Movie theater: Theater start: DateTime	Showtime	Creates a new showtime with one Seat for each seat in the theater, then adds it with Theater.addShowtime. Returns the new showtime, or null if it overlaps an existing one. The change appears on the website and kiosks within 1 second.
editShowtime	showtimeId: int newStart: DateTime	boolean	Changes a showtime's start time. Returns false and makes no change if the showtime already has tickets sold (Showtime.hasSoldTickets) or the new time overlaps another showtime (Showtime.conflictsWith).
removeShowtime	showtimeId: int	boolean	Removes a showtime using Theater.removeShowtime. Returns false if the showtime is not found or already has tickets sold.
moveMovie	movie: Movie from: Theater to: Theater	boolean	Moves the movie's upcoming showtimes from one theater to another. Each showtime is checked for conflicts

Operation	Parameters	Returns	Description
			in the new theater and for sold tickets. If any check fails, nothing is moved and the method returns false.
correctPurchase	purchaseId: int reason: String	boolean	Fixes a mistake in a confirmed purchase, such as a wrong ticket type or a double charge, by refunding or reissuing the affected tickets. The reason is saved to the purchase log. Returns false if the purchase is not found.
generateSalesReport	movie: Movie start: Date end: Date	SalesReport	Creates a SalesReport for the movie over the given dates and calls its generate operation. Returns the finished report.

6.4.9 Feedback

A customer's answer to the optional rating prompt shown after checkout. The prompt uses a 1 to 5 smiley-face scale. Skipping the prompt does not affect the purchase, so a Purchase has at most one Feedback.

Related requirements: 3.2.7, 3.6.7; use case 3.3.1

Attributes

Attribute	Type	Description
feedbackId	int	Unique ID for the feedback.
purchase	Purchase	The purchase the feedback is about.
rating	int	Rating chosen by the customer, from 1 (very unhappy) to 5 (very happy).
submittedAt	DateTime	Time the feedback was submitted.

Operations

Operation	Parameters	Returns	Description
submit	rating: int	boolean	Checks the rating with isValidRating, then stores it with the current time in submittedAt. Returns false and stores nothing if the rating is not valid.
isValidRating	rating: int	boolean	Returns true if the rating is a whole number from 1 to 5.

6.4.10 SalesReport

A report of ticket sales and pricing for one movie over a date range, created by an administrator. Only CONFIRMED purchases are counted.

Related requirements: 3.2.8; use case 3.3.3

Attributes

Attribute	Type	Description
movie	Movie	The movie the report covers.
startDate	Date	First day included in the report.
endDate	Date	Last day included in the report.
ticketsSold	int	Total number of tickets sold for the movie in the date range.
totalRevenue	double	Total ticket sales in US dollars, after discounts.
averagePrice	double	Average price paid per ticket (totalRevenue ÷ ticketsSold). 0 if no tickets were sold.
salesByType	Map<TicketType, Integer>	Number of tickets sold for each ticket type (ADULT, STUDENT, SENIOR, MILITARY).

Operations

Operation	Parameters	Returns	Description
generate	none	void	Reads all confirmed purchases for the movie between startDate and endDate from the daily purchase logs, and fills in ticketsSold, totalRevenue, averagePrice, and salesByType. If there are no sales, every value is set to 0 (Section 3.2.8.5).
exportCsv	none	String	Returns the report as comma-separated text so administrators can open it in a spreadsheet.

6.5 Development Plan and Timeline

The plan covers the rest of the Fall 2026 semester, from October 9 until the final project report is due on December 14, 2026. All dates below are in 2026. Work is divided by the application services in Figure 6.1, so each team member owns complete services and the classes inside them. Because the services only talk to each other through internal API calls and the shared database, both members can work in parallel once the database schema is set. The remaining course deliverables are listed as tasks T14 to T19, so writing the SRS and building the system move forward together.

6.5.1 Partitioning of Tasks

ID	Task	Description	Owner	Dates
T1	Project setup and database schema	Set up the GitHub repository and development environment. Design the central database tables for all classes, and set up the daily off-site backup.	Both	Oct 9 – Oct 18
T2	Web server / API gateway	HTTPS endpoint, load balancing, CAPTCHA bot detection, virtual waiting queue,	Zachary	Oct 12 – Oct 25

		one-device session check, and WebSocket push of seat changes.		
T3	Catalog Service	Movie, Theater, and Showtime classes; seat maps; pulling and caching review scores from Rotten Tomatoes and IMDb.	Zachary	Oct 19 – Nov 1
T4	Account Service	CustomerAccount class: registration, confirmation e-mail, login/logout, loyalty points, and purchase history.	Ryoka	Oct 19 – Nov 1
T5	Seat Reservation Service	Seat class: 5-minute holds inside database transactions, releasing expired holds, and preventing double-selling.	Zachary	Oct 26 – Nov 8
T6	Checkout Service and payment integration	Purchase class: ticket types and discounts, 20-ticket limit, sales window, and connecting to credit card, PayPal, and Bitcoin processors.	Ryoka	Nov 2 – Nov 15
T7	Ticketing Service	Ticket class: signed unique tickets, e-mail delivery, kiosk printing with backup printer, and scanning at the entrance.	Ryoka	Nov 9 – Nov 22

T8	Administration Service	Administrator class: admin login, adding/editing/removing showtimes with conflict checks, moving movies, and correcting purchases.	Zachary	Nov 9 – Nov 22
T9	Feedback Service	Feedback class and the post-checkout rating prompt.	Ryoka	Nov 16 – Nov 22
T10	Reporting Service	SalesReport class, daily purchase logs, and CSV export.	Zachary	Nov 16 – Nov 22
T11	Website and kiosk user interface	Customer website, kiosk touchscreen layout, and admin console. Language support for English, Spanish, and Swedish.	Both	Oct 26 – Nov 29
T12	Integration and testing	Run the test cases from the test plan (T14): unit tests for each class, end-to-end tests of the use cases in Section 3.3, and load and security testing against Section 3.5.	Both	Nov 30 – Dec 6
T13	Prototype demo	Deploy the prototype to one test theater and the website, demo it with the client, and fix issues found.	Both	Dec 7 – Dec 13
T14	Test Plan (Asst 3)	Write the test plan and test cases for	Both	Due Oct 22

		the functions in Sections 3.2 and 6.4, and add it to this SRS.		
T15	Architecture with Data Management (Asst 4)	Add the database design and networking details to the software architecture.	Both	Due Nov 12
T16	Ethics and Communications (Asst 5)	Write the ethics and communications section for the project.	Both	Due Dec 3
T17	Final Project Report draft	Combine all sections into one SRS report and fix earlier feedback.	Both	Due Dec 6
T18	Final Project peer review	Review another group's draft, and apply the review comments we receive.	Both	Due Dec 10
T19	Final Project Report	Push the finished report to the group GitHub repository and submit the link.	Both	Due Dec 14

6.5.2 Team Member Responsibilities

Team Member	Responsibilities
Zachary Gass	Web server / API gateway (T2) Catalog Service: Movie, Theater, Showtime (T3) Seat Reservation Service: Seat (T5) Administration Service: Administrator (T8) Reporting Service: SalesReport (T10) Admin console screens (part of T11)
Ryoka Kobayashi	Account Service: CustomerAccount (T4) Checkout Service and payment processor integration: Purchase (T6) Ticketing Service: Ticket, e-mail and kiosk printing

Team Member	Responsibilities
	(T7) Feedback Service: Feedback (T9) Customer website and kiosk screens, language support (part of T11)
Both	Database schema and backups (T1) Integration and testing (T12) Prototype demo (T13) Course deliverables T14–T19, and keeping this SRS document up to date Reviewing each other’s code before it is merged into the main branch

6.5.3 Timeline

The timeline follows the CS 250 Fall 2026 course calendar. Work starts on Friday, October 9, 2026, and the final project report is due on Monday, December 14, 2026. Course due dates are listed as milestones, so the team can see how building the system lines up with each submission.

Phase	Dates	Tasks	Milestone
1. Setup and test plan	Oct 9 – Oct 25	T1, T2, T14	Asst 3: Test Plan due Oct 22. Database schema and repository ready.
2. Core services	Oct 19 – Nov 8	T3, T4, T5	Customers can browse showtimes, create an account, and hold seats.
3. Checkout and data management	Nov 2 – Nov 15	T6, T15	Asst 4: Architecture with Data Management due Nov 12. Customers can pay.
4. Tickets and admin	Nov 9 – Nov 22	T7, T8, T9, T10	Full purchase flow works end to end, and admin mode is complete.
5. User interface	Oct 26 – Nov 29	T11 (runs alongside phases 2–4)	Website, kiosk, and admin screens finished in all three languages. Lighter week for Thanksgiving break (Nov 26).
6. Testing, ethics, and	Nov 30 – Dec 6	T12, T16, T17	Asst 5 due Dec 3. Final

draft			Project Report draft due Dec 6.
7. Peer review and final report	Dec 7 – Dec 14	T13, T18, T19	Peer review due Dec 10. Final Project Report due Dec 14.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2